

Design Pattern Command

Encapsulando requisições como objetos em uma aplicação **WPF**
+ **C#** com **MVVM**



Sistema de Cadastro de Alunos
Situação de Aprendizagem · Design Patterns

O que veremos hoje

1

Definição & Problema

O que é o Command e por que ele existe

2

Estrutura

Participantes, interfaces e fluxo

3

Código explicado

RelayCommand, ViewModel e XAML passo a passo

4

Análise & Mercado

Vantagens, desvantagens e exemplos reais

O que é o Command?

*"Encapsular uma **solicitação como um objeto**, permitindo parametrizar clientes com diferentes requisições, enfileirar ou registrar operações e suportar operações que podem ser desfeitas."*

— Gang of Four, Design Patterns, 1994



Encapsula

A ação vira um **objeto independente**, não um método preso na UI



Desacopla

Quem **dispara** a ação não precisa saber **como** ela funciona



Suporta Undo

Facilita reverter operações — base do **Ctrl+Z** em editores

Sem o padrão — código acoplado

✗ Sem Command

```
// Lógica misturada com a tela! private void  
BtnAdicionar_Click( object s, RoutedEventArgs e) { //  
UI e regra de negócio juntas alunos.Add(new Aluno {  
Nome = txtNome.Text }); lstAlunos.Items.Refresh(); }
```

- ✗ UI acoplada à lógica de negócio
- ✗ Impossível testar sem abrir a janela
- ✗ Lógica duplicada em vários botões

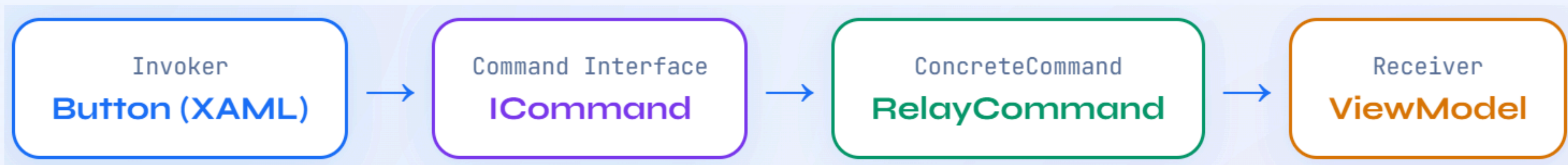
VS

✓ Com Command

```
// Botão só dispara – zero lógica aqui <Button  
Content="Adicionar Aluno" Command="{Binding  
AdicionarCommand}" /> // code-behind: só  
InitializeComponent()
```

- ✓ Separação total entre UI e lógica
- ✓ ViewModel testável de forma isolada
- ✓ Reutilizável em qualquer controle

Os 4 participantes do padrão



Button

O **Invoker**. Dispara o comando. Não sabe *o que* acontece depois.

ICommand

Interface com `Execute()` e `CanExecute()`. O **contrato** do padrão.

RelayCommand

Implementação de ICommand. Recebe actions via delegate — genérico e reutilizável.

ViewModel

O **Receiver**. Contém a lógica real. Métodos como `AdicionarAluno()`.



Models / Aluno.cs

```
// Model – representa a entidade do sistema public class
Aluno {
    public string Nome { get; set; }
}
```

O que é?

Um objeto **POCO** (Plain Old CLR Object) — simples, sem regras de negócio. Apenas armazena dados do aluno.

Princípio SRP

Single Responsibility Principle: o Model tem *uma única responsabilidade* — guardar o nome. Sem lógica, sem UI.

 Em sistemas maiores, o Model teria CPF, matrícula, notas... mas a responsabilidade continua sendo apenas armazenar dados.



Commands / RelayCommand.cs

```

public class RelayCommand : ICommand { // Guarda o método
    que será executado
    private readonly Action<object> _execute;
    // Valida se pode executar (opcional)
    private readonly Predicate<object> _canExecute;
    public RelayCommand( Action<object> execute,
        Predicate<object> canExecute = null) { _execute = execute;
        _canExecute = canExecute; } // Executa a ação passada como
        parâmetro
    public void Execute(object p) => _execute(p);
    // Se não tem validação, sempre permite
    public bool CanExecute(object p) => _canExecute = null ||
        _canExecute(p);
    // WPF reavalia o botão automaticamente public event
    EventHandler CanExecuteChanged { add =>
        CommandManager.RequerySuggested += value; remove =>
        CommandManager.RequerySuggested -= value; } }

```

_execute (Action)

Recebe o **método real** a executar como delegate. Ex: o método AdicionarAluno do ViewModel.

_canExecute (Predicate)

Validação **opcional**. Ex: "só remove se tiver seleção". Se não informada, sempre executa.

CanExecuteChanged

Avisa o WPF para **habilitar ou desabilitar** o botão automaticamente. Zero código extra na UI.

 O grande ponto: **métodos viram objetos** – esse é o coração do Command Pattern!

ViewModels / MainViewModel.cs

```
public class MainViewModel : INotifyPropertyChanged { //
    Lista que atualiza a UI automaticamente
    public ObservableCollection<Aluno> Alunos { get; }
    // Ligado ao TextBox da tela public string NomeAluno { get;
    set; } // Controla o item selecionado na lista public Aluno
    AlunoSelecionado { get; set; } // ★ COMMAND PATTERN em
    ação
    public ICommand AdicionarCommand { get; }
    public ICommand RemoverCommand { get; }
    public MainViewModel() { Alunos = new
    ObservableCollection<Aluno>(); // Passando métodos como
    objetos! AdicionarCommand = new
    RelayCommand(AdicionarAluno); RemoverCommand = new
    RelayCommand(RemoverAluno, PodeRemover); } private void
    AdicionarAluno(object obj) => Alunos.Add(new Aluno { Nome =
    NomeAluno }); private void RemoverAluno(object obj) =>
    Alunos.Remove(AlunoSelecionado); private bool
    PodeRemover(object obj) => AlunoSelecionado != null; }
```

INotifyPropertyChanged

Permite que a tela seja **atualizada automaticamente** quando uma propriedade muda. Base do MVVM.

ObservableCollection

Diferente de List: **notifica a UI** ao adicionar ou remover. A ListBox atualiza sozinha, sem código.

PodeRemover()

Passada como validação ao RelayCommand — o botão **Remover fica desabilitado** automaticamente se nada selecionado.

 O ViewModel é o *coração* da aplicação — toda lógica vive aqui, sem tocar na UI.



Views / MainWindow.xaml

```
←!— 1. Conecta a View ao ViewModel →  
<Window.DataContext> <vm:MainViewModel/>  
</Window.DataContext> ←!— 2. TextBox ligado à propriedade  
→ <TextBox Text="{Binding NomeAluno,  
UpdateSourceTrigger=PropertyChanged}" /> ←!— 3. 🔥  
COMMAND PATTERN em ação →  
<Button Content="Adicionar" Command="{Binding  
AdicionarCommand}" />  
<Button Content="Remover" Command="{Binding  
RemoverCommand}" />  
←!— 4. Lista que atualiza sozinha → <ListBox  
ItemsSource="{Binding Alunos}" SelectedItem="{Binding  
AlunoSelecionado}" />
```

1 DataContext

Liga a janela ao ViewModel. Tudo no ViewModel fica disponível para **Binding** na tela.

2 Text Binding

O TextBox escreve direto na propriedade NomeAluno do ViewModel. **Sem código no .cs.**

3 Command Binding ★

O botão **não tem evento Click**. Dispara o ICommand diretamente. Esse é o padrão em ação!

4 Zero Code-Behind

MainWindow.xaml.cs só tem InitializeComponent().
Separação **total**.

Como tudo se conecta

Quando o usuário clica em "Adicionar Aluno":



A View **nunca**
conhece a lógica de
negócio

O ViewModel **nunca**
conhece a interface
gráfica

O Command é a **ponte**
segura entre os dois
mundos

Como apresentar tecnicamente

Sobre a arquitetura

"O projeto segue o padrão **MVVM**, separando responsabilidades entre **Model** (dado), **View** (tela) e **ViewModel** (lógica)."

Sobre o Command Pattern

"Ao invés de usar eventos diretamente na interface, **encapsulei as ações em comandos**, promovendo desacoplamento e aderência ao padrão MVVM."

Sobre os benefícios

"Essa abordagem **melhora a manutenção**, facilita testes unitários e evita código misturado no code-behind — seguindo as boas práticas de engenharia de software."

Prós e Contras

✓ Vantagens

- ✓ Baixo acoplamento entre UI e lógica de negócio
- ✓ Código reutilizável — mesmo comando em vários controles
- ✓ ViewModel totalmente testável sem abrir a janela
- ✓ Separação de responsabilidades clara (princípio SRP)
- ✓ Suporte nativo no WPF via interface ICommand
- ✓ Botão desabilita automaticamente com CanExecute

⚠ Desvantagens

- ⚠ Mais classes para criar (RelayCommand, ViewModel)
- ⚠ Complexidade inicial maior que eventos simples
- ⚠ Pode parecer verboso em projetos muito pequenos
- ⚠ Curva de aprendizado para quem não conhece MVVM

💡 Em projetos reais e maiores, o overhead inicial compensa rapidamente em organização.

Onde o Command é usado na prática



Sistemas Bancários

Transferências e estornos encapsulados como comandos com rollback possível



ERPs

SAP, Oracle: ações de negócio auditáveis e rastreáveis via Command



Game Development

Unity: inputs encapsulados com undo/redo e replay de partidas



Editores de Texto

Word, VS Code: cada digitação é um Command — base do Ctrl+Z



WPF / MAUI / Xamarin

O próprio .NET define ICommand como padrão em toda plataforma XAML



Pipelines CI/CD

Etapas de deploy encadeadas e reversíveis usando Command Pattern

O que aprendemos



Command **encapsula ações como objetos**, desacoplando quem dispara de quem executa



Em WPF + MVVM, **ICommand / RelayCommand** é a implementação natural e nativa do padrão



O resultado é código **testável, organizado, desacoplado e de fácil manutenção**

ICommand

RelayCommand

MVVM

Desacoplamento

Button → ICommand → RelayCommand → ViewModel → Método



Obrigado!

Design Pattern **Command** aplicado com C# e WPF

GoF · Behavioral

Gang of Four

C# · WPF · MVVM

- 📁 Commands / RelayCommand.cs
- 📁 Models / Aluno.cs
- 📁 ViewModels / MainViewModel.cs
- 📁 Views / MainWindow.xaml